

Counselor Workflow Agent — Product and Build Report

1) What we are building

We are building a **counselor-facing workflow agent** for private / independent college counselors.

This is **not** a student chatbot, not a general “college admissions AI,” and not a tool that replaces counselor judgment. Independent educational consultants help students and families navigate the process of finding, applying to, and choosing schools that fit the student’s academic, social, and personal needs, which means the product has to support a long, multi-step, high-trust workflow.

The product thesis is:

The agent should take over the repetitive operating layer of counseling so the counselor can spend more time on strategy, nuance, and relationship-heavy advising.

2) Why this product matters

The strongest target segment is **private / independent counseling**, not general public-school counseling. In the NACAC/EAB counselor survey, public-school counselors reported spending about **22%** of their time on postsecondary admission counseling, while private-school counselors reported **51%**. That means a workflow-efficiency product has more usable surface area in private counseling environments, where more of the workday is actually spent on admission-related work.

3) Core product rule

The agent should handle tasks that are:

- repeated across many students,
- structured or semi-structured,
- document-heavy,
- deadline-driven,
- low-to-medium judgment,
- valuable even if a counselor still reviews them.

The agent should **not** own tasks that depend heavily on:

- final admissions strategy,
- emotional nuance,
- ethical judgment,
- student voice,
- sensitive family dynamics,
- or official submission without review.

This matters because IECA's guidance is explicit that the student should be in charge of the process, application materials should be the student's own work, and consultants should not write or heavily edit student essays or complete forms on the student's behalf.

4) What work counselors likely spend the most repetitive time on

There does not appear to be a public time-and-motion study that cleanly breaks an independent counselor's week into micro-tasks, so the percentages below are a product estimate based on counselor workload research, Common App workflow complexity, professional counseling descriptions, and current counselor software. That estimate is:

A. Coordination, accountability, and communication — roughly 25% to 30%

This includes scheduling, reminders, chasing missing items, parent updates, student nudges, next-step tracking, and keeping everyone aligned. IECA explicitly describes tailored timelines, accountability, organization, and parent communication as part of the consultant role, and current counselor software is built heavily around those functions.

B. Document and application operations — roughly 25% to 30%

This includes profile intake, requirement tracking, recommendation logistics, missing-material checks, application section review, and submission readiness. Common App's first-year workflow requires legal/personal info, testing information, supporter coordination, college-specific requirements, and multiple application components, which creates a large, structured operations burden.

C. College-list research and routine planning — roughly 15% to 20%

This includes first-pass school research, comparing requirements, building initial shortlists, checking deadlines and testing policies, and assembling the first version of a plan. Counselors routinely evaluate curriculum fit, admission requirements, costs, scholarships, financial aid, majors, and deadlines.

D. Meeting prep, memory, and internal synthesis — roughly 10% to 15%

This is the work of remembering where a student is, what changed, what is blocked, and what the counselor needs to address next. Current counselor platforms emphasize notes and recurring profile documentation, which is a strong signal that this repeated memory work is real and persistent.

E. Essay-process operations, not final essay judgment — roughly 10% to 15%

This includes prompt tracking, version organization, feedback consolidation, and consistency checking across essays, activities, and the rest of the application. The application process itself creates recurring writing management work, but the agent must stop short of ghostwriting.

F. High-judgment human work — roughly 15% to 20%

This includes positioning strategy, difficult tradeoffs, sensitive advising, final school-list judgment, and deep essay direction. This is where the counselor remains essential.

Bottom line: my working estimate is that **about 50% to 65% of the counseling workflow is meaningfully AI-augmentable**, but mostly at the operations layer rather than the strategy layer. That is an internal product estimate based on the sources above, not a published benchmark.

5) Product goal

The product goal is to reduce counselor time spent on:

- remembering student state,
- manually organizing scattered information,
- repetitive drafting,
- requirement tracking,
- follow-up coordination,
- repeated application assembly checks.

The product should create value in three ways:

1. **Memory** — the agent remembers the student so the counselor does not have to.
2. **Orchestration** — the agent keeps the workflow moving without manual chasing.
3. **Preparation** — the agent turns raw materials into counselor-ready outputs.

6) What the agent should cover

6.1 Meeting prep and post-meeting support

This should be a core MVP feature.

Before a meeting, the agent should:

- summarize student profile,
- identify current phase and milestone status,
- show what changed since the last meeting,
- list open tasks and blockers,
- highlight upcoming deadlines,
- generate suggested agenda items,
- flag inconsistencies or risks.

After a meeting, the agent should:

- generate a structured summary,
- extract decisions,
- assign action items,
- assign due dates,
- update the student case file,
- prepare follow-up drafts to student and parents.

This is strongly aligned with workflow-agent design patterns: read records, summarize state, update records, and propose next actions with human review.

6.2 Student profile intake and synthesis

The agent should ingest:

- questionnaires,
- transcripts,
- test scores,
- activity lists,
- counselor notes,
- essay drafts,
- parent context,

- school preferences.

It should convert them into a structured case file with:

- identity / theme summary,
- academics,
- testing,
- activities,
- school-list preferences,
- essay status,
- recommendation status,
- deadlines,
- risks / gaps,
- open questions.

This is one of the highest-ROI functions because it converts messy input into usable counselor context.

6.3 College-list and requirement support

The agent should:

- build first-pass school comparison tables,
- assemble school-specific requirement summaries,
- show testing policies,
- surface application deadlines,
- compare cost / scholarship / major-fit information,
- help identify missing research areas.

Counselors repeatedly evaluate curriculum fit, admission requirements, costs, scholarships, financial aid, majors, and deadlines, so this is a strong candidate for structured AI assistance.

6.4 Activity-list support

The agent should:

- normalize raw activity inputs,

- suggest missing details,
- flag vague descriptions,
- check consistency across résumé, activities, and essays,
- propose stronger structure for activity descriptions using only student-provided facts,
- maintain version history.

This must be built with strong factual guardrails: no inflated claims, no invented metrics, no unsupported language.

6.5 Testing support

The agent should:

- gather school testing requirements,
- build testing timelines,
- maintain score records,
- create study-plan drafts,
- track English-proficiency requirements for international students,
- flag when testing strategy conflicts with school list or scholarship goals.

Common App and current counselor practice make testing policy tracking a recurring structured task.

6.6 Recommendation workflow

The agent should:

- help compare recommenders,
- draft brag-sheet templates from student data,
- draft personalized request emails,
- track send dates and follow-up dates,
- monitor completion status against deadlines.

This is repetitive, structured, and very workflow-friendly.

6.7 Application assembly and submission readiness

The agent should:

- map student data into structured application fields,
- prepare Common App-compatible sections for review,
- check for missing fields,
- detect contradictions across documents,
- track school-specific supplements,
- maintain a readiness dashboard,
- flag submission blockers.

Common App clearly requires structured personal data, testing information, college-specific requirements, and coordinated materials, so application assembly is a major automation surface.

6.8 Parent and student communication drafting

The agent should:

- draft weekly updates,
- summarize completed work,
- explain upcoming priorities,
- prepare reminder messages,
- maintain tone templates by counselor,
- support optional review-before-send workflows.

This should begin as **draft-first**, not full autonomy.

6.9 Post-submission follow-up

The agent should:

- track confirmations,
- remind users to check portals,
- watch for missing materials,
- track follow-up actions,
- draft thank-you notes,
- maintain a submission log.

7) What the agent should not do

The agent should not:

- make final school-list decisions,
- independently determine admissions strategy,
- write or heavily edit essays,
- submit applications without explicit review,
- send sensitive communications without permissions and safeguards,
- generate unsupported claims in activities or application materials,
- act as if it “knows” the student better than the counselor.

This is both a trust issue and an ethics issue. IECA's published standards make student ownership and authenticity especially important.

8) How to think about Common App and form entry

The team should not start by trying to build an uncontrolled bot that “fills Common App by itself.”

Instead, build this in layers:

Layer 1

Create a **structured application data model** that stores student information once and maps it to likely application fields.

Layer 2

Create a **review-ready application packet**:

- personal info,
- education,
- activities,
- testing,
- school-specific requirements,
- uploads needed.

Layer 3

Add **assisted entry**:

- copy-ready fields,

- browser-assisted filling,
- visible confirmation screens,
- human approval for each section.

Layer 4

Only later consider deeper integration or more automation if legally, technically, and operationally sound.

This approach fits OpenAI's guidance to start with well-defined tools, predictable workflows, and human oversight for higher-risk actions.

9) How the system should be built

The product should not be a single chat box. It needs a workflow system underneath it.

9.1 Recommended architecture

Use:

- one **manager / orchestrator agent**,
- structured tools underneath,
- rule-based checks,
- retrieval over student records and policy documents,
- human review gates,
- event-driven updates,
- audit logging.

OpenAI's agent guidance recommends clear orchestration, simple tool-connected workflows, prompt templates, and careful governance rather than overcomplicated autonomy.

9.2 Core system objects

The team should define these first:

- Student
- Parent / Guardian
- Counselor
- Case File

- Milestone
- Task
- School
- School Requirement
- Application Section
- Activity
- Essay Artifact
- Recommender
- Meeting
- Communication
- Submission Event
- Risk Flag
- Review Queue Item

Without a clean schema, the agent will become a messy prompt system instead of a real workflow product.

9.3 Core modules

Recommended modules:

- Intake / Profile Parser
- Case File Generator
- Milestone Engine
- Meeting Copilot
- Requirement Intelligence
- College List Assistant
- Activity Description Assistant
- Testing Assistant
- Recommendation Assistant
- Application Assembly Engine

- Communication Drafting Layer
- Review / Approval Queue
- Audit Log / Source Trace Layer

The product implication is important:

This cannot be a static checklist.

It needs a dynamic milestone engine that can change:

- sequencing,
- urgency,
- reminders,
- counselor alerts,
- recommended resources,
- and additional support steps.

But these documents should be used as examples, not as unquestioned ground truth.

10) How counselors will communicate with the agent

The counselor should communicate with the agent primarily through a **case-centric workflow interface**, not through a blank chatbot.

This is important because the product is not meant to be a general AI assistant. It is meant to function as a workflow operating layer inside the counselor's day-to-day process. That means the main interaction model should be built around **student records, milestones, tasks, documents, meeting prep, and approval flows**, with natural-language chat serving as a support layer rather than the core product.

10.1 Primary interaction model: student case file

The main interface should be the **student case file**.

When a counselor opens a student, the agent should be embedded directly into that student's workspace and should immediately surface:

- current phase / milestone status,
- what changed since the last meeting,
- open tasks and blockers,
- upcoming deadlines,

- missing materials,
- recommended next steps,
- recent notes and communications,
- flagged risks or inconsistencies.

The goal is for the counselor to instantly understand:

Who is this student, where are they in the process, what still needs to happen, and what should I focus on right now?

This is especially important because counselors manage many students at once and cannot realistically remember every student's detailed status without support.

10.2 Chat should exist, but as a secondary layer

The system should still include a natural-language chat interface, but chat should be **secondary**, not primary.

The counselor should be able to ask questions like:

- "Prepare me for today's meeting."
- "What are the biggest blockers for this student?"
- "What changed since the last session?"
- "Draft this week's parent update."
- "Show me missing Common App fields."
- "Compare this student's current school list with testing requirements."
- "Turn these notes into action items."

However, this chat should always be grounded in the student's case file, milestone history, and stored documents. It should not behave like a free-floating general chatbot.

The rule is:

Chat is a way to query and direct the workflow, not replace the workflow UI.

10.3 Meeting mode should be a core workflow

One of the most valuable interaction patterns should be a dedicated **meeting mode**.

Before a meeting, the counselor should be able to click:

- "Generate prep brief"

The agent should then provide:

- student profile summary,
- current milestone stage,
- recent progress,
- unfinished tasks,
- upcoming deadlines,
- previous meeting takeaways,
- suggested agenda items,
- potential discussion risks.

After a meeting, the counselor should be able to:

- paste rough notes,
- upload meeting notes,
- upload or paste a transcript,
- optionally record a voice memo later,
- click “Convert to summary + action items.”

The agent should then:

- generate a structured summary,
- extract decisions,
- assign next steps,
- assign owners and due dates,
- update the case file,
- draft follow-up communication.

This should become one of the system’s signature workflows.

10.4 The agent should also proactively communicate

The relationship should not be purely reactive.

The agent should proactively surface priority items to the counselor, such as:

- students needing attention today,
- overdue milestones,

- missing recommendation letters,
- upcoming application deadlines,
- missing documents,
- parent updates ready for review,
- students showing no progress this week,
- inconsistent information across application materials.

This means the agent should act partly like an operations chief of staff, not just a passive assistant.

Examples of proactive system prompts:

- “3 students need attention today.”
- “2 parent updates are ready for approval.”
- “1 student has an incomplete activities section.”
- “This application is missing testing data.”
- “A recommendation request follow-up is due tomorrow.”

10.5 Sensitive actions should go through an approval queue

Anything externally meaningful or high-stakes should go through a **review / approval queue**.

Examples:

- parent emails,
- student reminders,
- recommender request emails,
- application field mappings,
- milestone updates,
- readiness summaries,
- any outbound communication,
- any action that changes official records.

The workflow should be:

1. agent prepares,

2. counselor reviews,
3. counselor edits if needed,
4. counselor approves,
5. system executes.

For MVP, the agent should mostly operate in:

- **Suggest**
- **Draft**
- **Execute with approval**

It should avoid full autonomous execution for sensitive actions.

10.6 Best UI model: hybrid, not chat-only

The best interface is likely a hybrid of:

- structured dashboards,
- student case pages,
- milestone trackers,
- task lists,
- approval cards,
- document panels,
- quick-action buttons,
- and a contextual chat panel.

This will work better than a pure chatbot because college counseling contains many recurring, structured workflows. The agent should feel like it is embedded into the workflow, not sitting outside of it.

10.7 Recommended communication channels

The recommended communication stack is:

Primary

- web app dashboard
- student case page
- meeting mode

- approval queue

Secondary

- email summaries
- calendar reminders
- optional notifications later

The main product should not initially rely on Slack, SMS, or a mobile-first chat experience.

10.8 Internal design principle

The clearest internal instruction for the tech team is:

The counselor should communicate with the agent mostly by working through student records, tasks, briefs, and approvals — with chat as a helper, not as the main product.

And the clearest product framing is:

The agent should feel less like talking to an AI, and more like working with an always-prepared chief of staff inside each student case.

11) Recommended MVP scope

MVP 1 — highest value, lowest risk

Build these first:

1. Student case file + profile ingestion
2. Meeting prep brief
3. Post-meeting summary + task extraction
4. Milestone tracker
5. Requirement tracker
6. Application readiness dashboard
7. Draft-only student / parent updates
8. Recommendation request workflow

This scope directly attacks the biggest repetitive workload buckets.

MVP 2 — deeper workflow intelligence

Add:

1. College-list comparison assistant
2. Activity description assistant
3. Testing strategy assistant
4. Document consistency checker
5. Essay-process operations layer
6. Personalization logic by profile type

MVP 3 — controlled automation and integrations

Add:

1. Assisted application form entry
2. Calendar / email integration
3. Parent portal
4. Student portal
5. Controlled auto-send rules for approved communication types
6. Analytics dashboard

12) Suggested division of work for the two builders

AI / CS lead

Own:

- orchestration,
- data schema,
- retrieval,
- structured output definitions,
- prompt policy system,
- evaluation harness,
- hallucination controls,
- source-attribution logic.

Agent-builder / implementation lead

Own:

- frontend / workflow interface,
- document upload flows,
- case-file UI,
- milestone dashboard,
- approval queue,
- notification mechanics,
- browser-assist flows,
- system usability.

Shared

Both should jointly define:

- event model,
- milestone rules,
- approval rules,
- test cases,
- counselor feedback loops,
- failure handling.

13) Product guardrails

Every externally meaningful output should show:

- source basis,
- confidence / uncertainty,
- last updated time,
- whether human review is required.

Every automation should have one of three modes:

- Suggest
- Draft

- Execute with approval

For MVP, almost everything should live in **Suggest** or **Draft**.

14) Success metrics

The team should optimize for operational outcomes, not “wow demo” outputs.

Track:

- hours saved per counselor per week,
- prep time per student meeting,
- time spent on follow-up/admin,
- time to assemble an application readiness view,
- draft acceptance rate,
- number of missed / late tasks,
- number of missing-item surprises before submission,
- edit distance between AI draft and final counselor output,
- weekly active usage by counselors,
- trust / satisfaction scores.

The most important KPI is:

hours of counselor time saved per week without reducing trust or quality

15) Final instruction to the team

Build this as a **counselor-in-the-loop workflow system**, not an “AI college counselor.”

The product should own:

- memory,
- coordination,
- preparation,
- tracking,
- drafting,
- consistency checking,
- and repetitive application operations.

The counselor should keep:

- strategy,
- nuance,
- ethics,
- student voice,
- and final judgment.

The clearest internal product sentence is:

We are building the operating system around a great counselor, not trying to replace the counselor.